

BEDROCK ARCHITECTURE

Bedrock Target Intrinsic

C Compiler Builtins and Header Interfaces

Draft

CONTENTS

1	Scope and Status	1
2	Compiler Interface Model	1
2.1	Names and Ownership	1
2.2	Header Topology and Exposure	1
2.3	Common Compilation Rules	2
3	Public Target Interface	2
3.1	Segment-Image Value Macros	2
3.2	Far Pointer Builtins	2
3.3	Core Builtins	3
3.4	Memory Builtins	4
3.5	Integer Builtins	4
3.6	Floating-Point Builtins	4
3.7	Approximate Transcendental Lowering Policy	5
4	System Target Interface	5
4.1	System-Register Builtins	5
4.2	Cache-Management Builtins	6
4.3	MMU Builtins	6
4.4	Processor-State Builtins	7
5	Shared Header Types	7

LIST OF TABLES

2-1	Target Intrinsic Header Families	1
3-1	Far Pointer Builtins	3
3-2	Core Builtins	3
3-3	Memory Builtins	4
3-4	Integer Builtins	4
3-5	Floating-Point Builtins	4
4-1	System-Register Builtins	5
4-2	Cache-Management Builtins	6
4-3	MMU Builtins	6
4-4	Processor-State Builtins	7
5-1	Target Intrinsic Shared Types	7

LIST OF FIGURES

1 SCOPE AND STATUS

This document defines the compiler-owned Bedrock C target interface: builtin names, implementation-reserved header wrappers, source signatures, lowering requirements, availability and privilege constraints, and compiler-visible effects. It is the contract shared by a Bedrock-aware C frontend, optimizer, backend, and the target headers.

Architectural instruction behavior remains defined by the Bedrock ISA Specification. The baseline calling convention is defined by the Bedrock C ABI. Far-pointer source types are defined by the Bedrock C Far-Pointer Extensions, while their representation and calls are defined by the C ABI. This document specifies how source programs reach those facilities; it does not duplicate their architectural or language semantics.

Interface name: `bedrock-target-intrinsics`

Source language: ISO C with implementation-reserved target interfaces

2 COMPILER INTERFACE MODEL

2.1 Names and Ownership

The compiler builtin namespace begins with `__builtin_bedrock_`. Each **Name** entry in this document denotes that prefix followed by the displayed name. For example, `cuid` denotes `__builtin_bedrock_cuid`. The ordinary header wrapper uses the `__bedrock_` prefix with the same name unless its entry or the far-pointer construction interface specifies a macro spelling.

All of these identifiers are implementation-reserved. A builtin is a compiler primitive, not an external function symbol. The target headers own wrapper declarations and umbrella include topology; this document owns their ABI-visible meaning.

2.2 Header Topology and Exposure

`<bedrockintrin.h>` is the public umbrella. It includes the far, core, memory, integer, and floating-point families. `<bedrocksystemintrin.h>` is the system umbrella for system-register, cache, MMU, and processor-state facilities. Any family header may also be included directly.

Table 2-1. Target Intrinsic Header Families

Family	Header	Umbrella	Exposure
far pointer	<code><bedrockfarintrin.h></code>	public	bedrock-c-far profile
core	<code><bedrockcoreintrin.h></code>	public	unprivileged core operations
memory	<code><bedrockmemoryintrin.h></code>	public	user ordering and access hints
integer	<code><bedrockintegerintrin.h></code>	public	nonportable integer operations
floating point	<code><bedrockfpintrin.h></code>	public	floating-point environment and classification
system registers	<code><bedrocksysregintrin.h></code>	system	architectural system-register access
cache	<code><bedrockcacheintrin.h></code>	system	cache management
MMU	<code><bedrockmmuintrin.h></code>	system	supervisor translation operations
processor state	<code><bedrockstateintrin.h></code>	system	runtime state management

2.3 Common Compilation Rules

Family wrappers are static inline functions unless an encoded immediate or type operand requires a macro. Every builtin listed by an enabled profile must be recognized by the compiler. If a required optional feature is disabled and no fallback is specified here, compilation fails rather than emitting an unresolved call.

An encoded immediate must be an integer constant expression in the stated range. Header inclusion neither suppresses nor emulates architectural privilege checks. An operation is volatile only when its entry states an externally observable architectural effect. Compiler memory effects and hardware ordering are exactly those stated by the entry; one does not imply the other.

The **C interface** column uses `result(arguments)` shorthand. Exact declarations are in the named header; `u8`, `u16`, `u32`, and `u64` abbreviate the corresponding `uintN_t` types, and `result` denotes `__bedrock_query_result_t`. A clobber means a compiler memory clobber, while a named fence also has the hardware behavior defined by the ISA.

3 PUBLIC TARGET INTERFACE

3.1 Segment-Image Value Macros

Header: `<bedrockfarintrin.h>`

The public far-pointer header defines the pure value macros `__BEDROCK_SEGMENT_IMAGE`, `__BEDROCK_SEGMENT_IMAGE_FOR_BASE`, and `__BEDROCK_SEGMENT_DISABLED`. They construct 64-bit segment images without reading or writing architectural state and are available through `<bedrockintrin.h>`.

`__BEDROCK_SEGMENT_IMAGE(base_page, exponent, mantissa, bounds_only)` places the low 52, 5, and 6 bits of the first three operands in their architectural fields and normalizes `bounds_only` to zero or one. Each operand is evaluated once. An ABI-valid enabled image uses an exponent from 0 through 31 and a mantissa from 1 through 63. The macro only packs fields; it does not validate the resulting base, span, limit, or address canonicity.

`__BEDROCK_SEGMENT_IMAGE_FOR_BASE(base, ...)` encodes `[(uint64_t)base/4096]` in the base-page field. It discards the low 12 bits, so a misaligned operand names the containing page and does not preserve the original byte address. The macro imposes no alignment precondition and does not adjust a later offset to compensate for the discarded bits. `__BEDROCK_SEGMENT_DISABLED` is the canonical all-zero disabled image. ABI-visible disabled images use that constant rather than constructing an image with a zero mantissa and nonzero remaining fields.

3.2 Far Pointer Builtins

Header: `<bedrockfarintrin.h>`

The four construction operations use the macro spellings `__BEDROCK_FAR_PTR_INIT`, `__BEDROCK_FAR_FLAT_PTR_INIT`, `__BEDROCK_FAR_NULL`, and `__BEDROCK_FAR_PTR_FROM_SEGMENT`. Their pointer-type operand is handled by the compiler and is not a runtime C value.

`__BEDROCK_FAR_PTR_FROM_SEGMENT` accepts an enabled translated source image with nonzero mantissa and bounds-only mode clear.

It adds the encoded base to the supplied offset modulo 2^{64} . A zero sum denotes null, but the converted bounds-only image is retained for zero and nonzero sums alike. It performs no overflow test. An already-linear address and bounds-only image instead use `__BEDROCK_FAR_PTR_INIT`; a flat address uses `__BEDROCK_FAR_FLAT_PTR_INIT`.

Table 3-1. Far Pointer Builtins

Name	C interface	Lowering	Constraint and effect
<code>far_ptr_init</code>	<code>far</code> <code>T(type, address, image)</code>	value construction	Far type required; constant operands preserve constant-expression status; no fault, segment write, or memory access.
<code>far_flat_ptr_init</code>	<code>far T(type, address)</code>	value construction	Far type required; attaches the canonical disabled image without faulting.
<code>far_null</code>	<code>far T(type)</code>	constant construction	Far type required; produces the canonical all-zero representation.
<code>far_ptr_from_segment</code>	<code>far</code> <code>T(type, offset, image)</code>	canonicalization	Far type required; returns the low 64 bits of encoded base plus offset with the converted bounds-only image, including when the zero address denotes null; no overflow test or fault.
<code>far_address</code>	<code>uint64_t(pointer)</code>	address extraction	Accepts a far data or function pointer; pure extraction without validation or bounds probing.
<code>far_same_encoding</code>	<code>int(left, right)</code>	two-word comparison	Compatible far pointer types required; pure representation comparison.

3.3 Core Builtins

Header: `<bedrockcoreintrin.h>`

Table 3-2. Core Builtins

Name	C interface	Lowering	Availability, constraint, and effect
<code>cpuid</code>	<code>u64(u64 selector)</code>	CPUID	User allowed; reads discovery state and not C memory.
<code>read_status</code>	<code>uint16_t(void)</code>	RDSTATUS	Unprivileged; reads STATUS and not C memory.
<code>rdpmc</code>	<code>uint64_t(counter_id)</code>	RDPMC	Unprivileged; ID is a constant in 0 through 65535; zero and unavailable IDs fault with <code>INVALID_SELECTOR</code> .
<code>breakpoint</code>	<code>void(void)</code>	BKPT	Any privilege; always raises the architectural breakpoint/debug exception.
<code>trace</code>	<code>void(marker)</code>	TRACE	Unprivileged; marker is a constant in 0 through 65535; emits a marker when enabled and otherwise acts as a NOP.
<code>yield</code>	<code>void(void)</code>	YIELD	Any privilege; scheduling hint, not a compiler or memory barrier.
<code>wait</code>	<code>void(void)</code>	WAIT	Unprivileged PAUSE-like convergent hint; retires normally and permits a finite implementation-selected delay, including zero, before subsequent execution.

3.4 Memory Builtins

Header: <bedrockmemoryintrin.h>

Table 3-3. Memory Builtins

Name	C interface	Lowering	Constraint and effect
<code>read_fence</code>	<code>void(void)</code>	<code>RFENCE</code>	Compiler and hardware read-ordering barrier.
<code>write_fence</code>	<code>void(void)</code>	<code>WFENCE</code>	Compiler and hardware write-ordering barrier.
<code>address_fence</code>	<code>void(void)</code>	<code>AFENCE</code>	Full compiler memory clobber and hardware address fence.
<code>nontemporal_store_u8</code>	<code>void(u8 *,u8)</code>	<code>MOVNT.B</code>	One writable byte; one hinted store, neither volatile nor a fence.
<code>nontemporal_store_u16</code>	<code>void(u16 *,u16)</code>	<code>MOVNT.W</code>	One writable 16-bit object; one hinted store, neither volatile nor a fence.
<code>nontemporal_store_u32</code>	<code>void(u32 *,u32)</code>	<code>MOVNT.L</code>	One writable 32-bit object; one hinted store, neither volatile nor a fence.
<code>nontemporal_store_u64</code>	<code>void(u64 *,u64)</code>	<code>MOVNT.Q</code>	One writable 64-bit object; one hinted store, neither volatile nor a fence.

3.5 Integer Builtins

Header: <bedrockintegerintrin.h>

Table 3-4. Integer Builtins

Name	C interface	Lowering	Result and effect
<code>clmul_u8</code>	<code>u8(u8,u8)</code>	<code>CLMUL.B</code>	Pure; low 8 bits of the carryless product.
<code>clmul_u16</code>	<code>u16(u16,u16)</code>	<code>CLMUL.W</code>	Pure; low 16 bits of the carryless product.
<code>clmul_u32</code>	<code>u32(u32,u32)</code>	<code>CLMUL.L</code>	Pure; low 32 bits of the carryless product.
<code>clmul_u64</code>	<code>u64(u64,u64)</code>	<code>CLMUL.Q</code>	Pure; low 64 bits of the carryless product.

3.6 Floating-Point Builtins

Header: <bedrockfpuintrin.h>

Table 3-5. Floating-Point Builtins

Name	C interface	Lowering	Availability, constraint, and effect
<code>read_fstatus</code>	<code>uint16_t(void)</code>	<code>RDFSTATUS</code>	Unprivileged; access to the floating-point environment.
<code>write_fstatus</code>	<code>void(uint16_t)</code>	<code>WRFSTATUS</code>	Unprivileged; reserved bits follow the ISA; changes the floating-point environment.
<code>read_fflags</code>	<code>uint16_t(void)</code>	<code>RDFFLAGS</code>	Unprivileged; reads accrued floating-point exception flags.
<code>write_fflags</code>	<code>void(uint16_t)</code>	<code>WRFFLAGS</code>	Unprivileged; reserved bits follow the ISA; writes accrued flags.

Name	C interface	Lowering	Availability, constraint, and effect
<code>fclass_f32</code>	<code>uint16_t(float)</code>	<code>FCLASS.S</code>	FPU required; pure ten-class one-hot classification; does not update <code>FFLAGS</code> .
<code>fclass_f64</code>	<code>uint16_t(double)</code>	<code>FCLASS.D</code>	FPU required; pure ten-class one-hot classification; does not update <code>FFLAGS</code> .

3.7 Approximate Transcendental Lowering Policy

A strict language-library operation shall not be lowered to an `FPTRANSA` instruction. A compiler may select `FPTRANSA` only when the source uses a separately defined approximate interface or an active compiler mode permits approximate mathematics. The selected instruction remains subject to its ISA accuracy, special-value, and floating-point exception contract.

4 SYSTEM TARGET INTERFACE

The compiler emits the requested architectural operation; runtime privilege and policy checks remain active.

4.1 System-Register Builtins

Header: `<bedrocksysregintrin.h>`

Table 4-1. System-Register Builtins

Name	C interface	Lowering	Availability, constraint, and effect
<code>write_status</code>	<code>void(uint16_t)</code>	<code>WRSTATUS</code>	Supervisor; reserved and privilege-controlled bits follow the ISA; compiler-visible side effect.
<code>read_control_register</code>	<code>uint64_t(selector)</code>	<code>RDCR</code>	Supervisor; selector is a constant in 0 through 65535 naming a defined register.
<code>write_control_register</code>	<code>void(selector,u64)</code>	<code>WRCR</code>	Supervisor; defined constant selector required; writes the register and clobbers memory.
<code>read_segment_register</code>	<code>uint64_t(selector)</code>	<code>RDSEG</code>	Unprivileged; selector is a compile-time <code>__bedrock_segment_register_t</code> .
<code>read_code_segment</code>	<code>uint64_t(void)</code>	<code>RDSEG CS</code>	Unprivileged; reads the current CS image through the dedicated source-only form.
<code>write_segment_register</code>	<code>void(selector,image)</code>	<code>WRSEG</code>	Unprivileged; validates the SREG image, changes address interpretation, and fully clobbers memory.

The system-register header defines `__bedrock_control_register_t` constants for every published control-register selector. The constants are names for the architectural selector values; they do not weaken the privilege, reserved-bit, or per-register validation performed by `RDCR` and `WRCR`.

4.2 Cache-Management Builtins

Header: <bedrockcacheintrin.h>

All cache range operations evaluate the address once and obtain the maintenance granule from CPUID `CACHE_TOPOLOGY`. A zero length emits no cache-maintenance instruction. A nonzero, non-wrapping byte range selects every maintenance block that intersects the range and lowers to one block instruction per selected block. Each instruction translates its own block address. If a later instruction faults, blocks completed by earlier instructions remain completed.

For a nonzero length, the mathematical half-open range $[\text{address}, \text{address} + \text{length})$ shall lie within the 64-bit address space. Otherwise the behavior of the call is undefined and no architectural cache-maintenance sequence is required. An implementation may diagnose a provably wrapping constant range. A valid range end is calculated mathematically, not modulo 2^{64} .

Table 4-2. Cache-Management Builtins

Name	C interface	Lowering	Availability and effect
<code>flush_dcache</code>	<code>void(void *,size_t)</code>	FLSHDCACHE	Supervisor; applies FLSHDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.
<code>invalidate_dcache</code>	<code>void(void *,size_t)</code>	INVDCACHE	Supervisor; applies INVDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.
<code>invalidate_icache</code>	<code>void(void *,size_t)</code>	INVICACHE	Supervisor; applies local INVICACHE to every CPUID maintenance block intersecting the range, serializes instruction fetch, and fully clobbers memory.
<code>writeback_dcache</code>	<code>void(void *,size_t)</code>	WRBKDCACHE	Supervisor; applies WRBKDCACHE to every CPUID maintenance block intersecting the range and fully clobbers memory.
<code>sync_cache</code>	<code>void(void *,size_t)</code>	SYNCCACHE	Supervisor; applies SYNCCACHE to every CPUID maintenance block intersecting the range, serializes instruction fetch, and fully clobbers memory.

4.3 MMU Builtins

Header: <bedrockmmuintrin.h>

Table 4-3. MMU Builtins

Name	C interface	Lowering	Constraint and effect
<code>invalidate_tlb</code>	<code>void(void)</code>	INVTLB	Supervisor; translation-state side effect and full memory clobber.
<code>invalidate_page</code>	<code>void(const void *)</code>	INVPAGE	Supervisor; argument supplies the linear page address and the operation fully clobbers memory.

Name	C interface	Lowering	Constraint and effect
<code>invalidate_asid</code>	<code>void(asid)</code>	INVASID	Supervisor; ASID is a constant in 0 through 65535 and the operation fully clobbers memory.
<code>switch_page_table</code>	<code>void(uint64_t ptc_r)</code>	SWPT	Supervisor; valid page-table image required; changes address space and fully clobbers memory.
<code>switch_page_table_asid</code>	<code>void(uint64_t,uint16_t)</code>	SWPTA	Supervisor; valid image and low 16-bit ASID; changes address space and fully clobbers memory.
<code>virtual_to_physical</code>	<code>result(address)</code>	VTOP	Supervisor; wrapper returns value and operation-defined <code>FLAGS</code> captured atomically; page walk without access to the translated C object.
<code>page_table_query</code>	<code>result(level,address)</code>	PTQUERY	Supervisor; level is an integer constant expression in 1 through 5; wrapper atomically returns value and <code>FLAGS</code> ; page walk without access to the queried C object.

4.4 Processor-State Builtins

Header: `<bedrockstateintrin.h>`

Table 4-4. Processor-State Builtins

Name	C interface	Lowering	Constraint and effect
<code>save_processor_state</code>	<code>void(void *area)</code>	SAVE	Writable, CPUID-sized, 4096-byte-aligned area; writes architectural state and fully clobbers memory.
<code>restore_processor_state</code>	<code>void(const void *area)</code>	RESTORE	Readable, feature-valid, 4096-byte-aligned area; reads the complete selected state image; may define <code>R0--R15</code> , <code>FLAGS</code> , valid GS registers, and extension state; fully clobbers memory and prevents TLS or segmented memory accesses from moving across a restored GS addressing context.

Both operations are unprivileged, but restoration of `STATUS` remains privilege-restricted. The backend must model every restored register and the full memory and addressing-context clobber explicitly. It must not treat restore as an opaque call or as only a memory effect.

5 SHARED HEADER TYPES

Table 5-1. Target Intrinsic Shared Types

Type	ABI contract
<code>__bedrock_segment_register_t</code>	selectors for DS, SS, and GS0 through GS5; CS instead uses the fixed RDSeg CS source form
<code>__bedrock_control_register_t</code>	constants for the architectural RDCR and WRCR selector namespace
<code>__bedrock_query_result_t</code>	u64 value; u16 flags; u16 reserved[3]